



**ClaimGuard**  
EVIDENCE BEFORE SUBMISSION

# Source Architecture and Data Flow Documentation

Evidence-first healthcare claim pre-validation

---

## Team Claimix

Oussema Harrabi

Wassim Hajji

Eya Ayedi

Ghassen Benkaji

Maram Kouki

CSTAM 3.0 - Velodoc Challenge  
First submission technical report

01 October 2026

**Scope statement.** ClaimGuard reviews synthetic claim packages against fictional payer rules before submission. It does not approve, deny, price, pay, diagnose, or submit a claim.

# Abstract

---

Healthcare claim preparation requires information from patient, encounter, coverage, provider, diagnosis, authorization, attachment, and line-item sources to agree before a payer receives the package. A missing identifier, inconsistent date, unsupported service, duplicate line, absent authorization, or arithmetic mismatch can stop a claim even when the underlying care is legitimate. The operational problem is therefore not only detection. A reviewer must also understand what failed, where the evidence came from, what can be corrected, and what must remain uncertain.

ClaimGuard is a tenant-scoped pre-submission quality gate for this workflow. It normalizes supported synthetic JSON, relational CSV, and FHIR R4 plus verified-sidecar packages into a strict 17-key claim envelope. A deterministic engine evaluates fifteen versioned fictional payer rules and produces one structured result for every rule. Each result carries its rule identity, severity, evidence pointers, explanation, corrective action, confidence semantics, and human-review requirement. An optional small-language-model layer may clarify a finding and recommend a next step, but only behind a deterministic scope guard, verifier, one repair attempt, and a labelled fallback. The model cannot change the authoritative result.

The system places the reviewer in control. Decisions append with actor and reason, corrections create a new immutable claim version, and run and decision events enter a SHA-256 hash chain. This report presents the problem, solution boundary, source architecture, end-to-end data flow, sequence behavior, logical database design, audit construction, implementation decisions, verified Phase 1 coverage, limitations, and planned extensions.

**Keywords:** healthcare claims, pre-validation, deterministic rules, explainable AI, human review, multi-tenancy, audit trail, FHIR R4, structured evidence.

# Contents

|                                                              |           |
|--------------------------------------------------------------|-----------|
| <b>Abstract</b>                                              | <b>i</b>  |
| <b>1 General Introduction</b>                                | <b>1</b>  |
| 1.1 Project context . . . . .                                | 1         |
| 1.2 Why architecture matters in this problem . . . . .       | 1         |
| 1.3 Report organization . . . . .                            | 1         |
| <b>2 Problem and Proposed Solution</b>                       | <b>2</b>  |
| 2.1 The operational problem . . . . .                        | 2         |
| 2.2 ClaimGuard’s response . . . . .                          | 2         |
| 2.3 Scope and non-goals . . . . .                            | 3         |
| <b>3 Requirements and Principles</b>                         | <b>4</b>  |
| 3.1 Phase 1 requirement coverage . . . . .                   | 4         |
| 3.2 Design principles . . . . .                              | 4         |
| 3.2.1 Deterministic authority . . . . .                      | 4         |
| 3.2.2 Evidence before prose . . . . .                        | 4         |
| 3.2.3 One internal contract . . . . .                        | 5         |
| 3.2.4 Versioning instead of overwriting . . . . .            | 5         |
| 3.2.5 Least authority for assistance . . . . .               | 5         |
| 3.2.6 Tenant scope at the service boundary . . . . .         | 5         |
| <b>4 Global Source Architecture</b>                          | <b>7</b>  |
| 4.1 Layer 1: clinic users and sources . . . . .              | 7         |
| 4.2 Layer 2: experience and access control . . . . .         | 8         |
| 4.3 Layer 3: ingestion and normalization . . . . .           | 9         |
| 4.4 Layer 4: deterministic validation core . . . . .         | 10        |
| 4.5 Layer 5: guarded assistance and human review . . . . .   | 11        |
| 4.6 Layer 6: versioned data, audit, and operations . . . . . | 12        |
| <b>5 Ingestion and Data Flow</b>                             | <b>13</b> |
| 5.1 The normalized claim contract . . . . .                  | 13        |
| 5.2 Flow interpretation . . . . .                            | 15        |
| 5.3 Failure behavior . . . . .                               | 15        |
| <b>6 Deterministic Rules and Structured Explainability</b>   | <b>16</b> |
| 6.1 Rule catalogue . . . . .                                 | 16        |
| 6.2 Five-state semantics . . . . .                           | 17        |
| 6.3 Structured result record . . . . .                       | 17        |

|           |                                                                 |           |
|-----------|-----------------------------------------------------------------|-----------|
| 6.4       | Explanation and correction recommendation . . . . .             | 17        |
| <b>7</b>  | <b>Reviewer Lifecycle and AI Controls</b>                       | <b>18</b> |
| 7.1       | One claim over time . . . . .                                   | 18        |
| 7.2       | The interactive assistant as a bounded graph . . . . .          | 20        |
| 7.3       | Why a small language model still needs this structure . . . . . | 20        |
| <b>8</b>  | <b>Logical Data and Audit Architecture</b>                      | <b>21</b> |
| 8.1       | Database organization . . . . .                                 | 21        |
| 8.2       | Important cardinalities and invariants . . . . .                | 23        |
| 8.3       | Tamper-evident audit chain . . . . .                            | 23        |
| <b>9</b>  | <b>Implementation and Verification</b>                          | <b>24</b> |
| 9.1       | Technology stack . . . . .                                      | 24        |
| 9.2       | Verification evidence . . . . .                                 | 24        |
| 9.3       | What the measurements do not prove . . . . .                    | 25        |
| <b>10</b> | <b>Limitations and Next Work</b>                                | <b>26</b> |
| 10.1      | Current limitations . . . . .                                   | 26        |
| 10.2      | Prioritized next work . . . . .                                 | 26        |
| <b>11</b> | <b>General Conclusion</b>                                       | <b>27</b> |
| <b>A</b>  | <b>Cross-Boundary Data Contracts</b>                            | <b>28</b> |
| A.1       | Seventeen-key claim envelope . . . . .                          | 28        |
| A.2       | Fifteen-key result record . . . . .                             | 29        |
| <b>B</b>  | <b>Workspace Responsibilities</b>                               | <b>30</b> |
|           | <b>Bibliography</b>                                             | <b>31</b> |

# List of Figures

|     |                                                                                                                                                            |    |
|-----|------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 3.1 | Global ClaimGuard source architecture. The plate is intentionally presented on a full landscape page to keep labels and trust boundaries readable. . . . . | 6  |
| 4.1 | Enlarged view of the clinic roles, supported sources, workspace, and access-control boundary. . . . .                                                      | 8  |
| 4.2 | Ingestion and normalization layer. Every accepted source converges on the 17-key scoring envelope. . . . .                                                 | 9  |
| 4.3 | Authoritative validation core. No model and no external network are required to establish a result. . . . .                                                | 10 |
| 4.4 | Guarded assistance and human review. The model path and reviewer authority remain separate. . . . .                                                        | 11 |
| 4.5 | Versioned storage and audit boundary. Review records are tenant-owned and history-preserving. . . . .                                                      | 12 |
| 5.1 | Claim package data flow from source through normalization, deterministic checks, review, versioning, and the audit ledger. . . . .                         | 14 |
| 7.1 | Sequence of claim submission, guarded explanation, reviewer decision, correction, and versioned recheck. . . . .                                           | 19 |
| 7.2 | Bounded assistant control graph. No node writes to the claim result or reviewer decision. . . . .                                                          | 20 |
| 8.1 | Logical PostgreSQL architecture and the principal relationships between tenant, run, result, workflow, assistant, and audit records. . . . .               | 22 |
| 8.2 | Audit events are linked by content-dependent SHA-256 digests. . . . .                                                                                      | 23 |

# List of Tables

|     |                                                                   |    |
|-----|-------------------------------------------------------------------|----|
| 3.1 | Phase 1 rubric coverage. . . . .                                  | 4  |
| 6.1 | Fictional payer rules evaluated for every accepted claim. . . . . | 16 |
| 8.1 | Principal data relationships. . . . .                             | 23 |
| 9.1 | Primary implementation technologies. . . . .                      | 24 |
| A.1 | Top-level scoring envelope and nested responsibilities. . . . .   | 28 |
| A.2 | Frozen result record emitted once per rule. . . . .               | 29 |
| B.1 | Role boundaries and workspace responsibilities. . . . .           | 30 |

# General Introduction

---

## 1.1 Project context

ClaimGuard was developed for the CSTAM 3.0 Velodoc challenge as a focused answer to one stage of the healthcare revenue-cycle workflow: checking a claim before it is submitted [1]. The platform is not a payer and it is not a clinical decision system. It sits between claim creation and payer submission and asks whether the administrative package is coherent enough to proceed to a human-controlled next step.

The first submission evaluates four technical capabilities. The system must ingest and normalize synthetic claim packages, apply ten to fifteen fictional payer rules, generate structured and explainable findings, and preserve an auditable history of checks, recommendations, confidence information, and decisions. The same submission also requires architecture and data-flow documentation. These requirements are tightly connected. Ingestion determines what the engine is allowed to know. The engine determines the finding. Explainability must cite the same source data. Audit must preserve how the outcome was produced and what the reviewer later did.

## 1.2 Why architecture matters in this problem

A conventional form-validation interface could detect missing values, but it would not be sufficient for a sensitive claim workflow. The system must distinguish a proven violation from missing evidence, prevent a generative model from relabelling a deterministic result, preserve the original input when a reviewer makes a correction, and ensure that one clinic cannot access another clinic's work. Those properties must come from system structure rather than from a prompt or an interface convention.

The architecture therefore separates six responsibilities: clinic users and sources, experience and access control, ingestion and normalization, deterministic validation, guarded assistance and human review, and versioned data with audit. The global figure is followed by dedicated sections because the most important guarantees are found at the boundaries between those layers.

## 1.3 Report organization

Chapter 2 defines the problem and the operational gap. Chapter 3 introduces ClaimGuard and maps the four Phase 1 requirements to the solution. Chapter 4 presents the global architecture. Chapters 5 through 9 examine access control, ingestion, deterministic validation, guarded assistance, review, persistence, and audit. Chapter 10 summarizes implementation and verification evidence. Chapter 11 states limitations and next work. The appendices reproduce the cross-boundary data contracts and principal API surfaces.

# Problem and Proposed Solution

---

## 2.1 The operational problem

A claim package is assembled from information that originates at different times and in different systems. Patient identity and coverage may come from registration. Provider and diagnosis data come from the clinical encounter. Service lines, quantities, prices, authorizations, and supporting documents accumulate during treatment. Before submission, those pieces must be internally consistent and must satisfy payer-specific administrative rules.

The cost of an error is not limited to rejection. An RCM reviewer must locate the defective value, understand the applicable rule, decide whether the issue is real, obtain missing information when necessary, and prove what was changed. A flat list of error codes creates another interpretation problem. A black-box generative answer is worse when it sounds convincing but cannot cite the original claim.

Five design challenges follow from this setting:

1. **Heterogeneous input.** JSON, tabular exports, and FHIR resources describe similar facts with different structures and levels of completeness.
2. **Semantic uncertainty.** Missing evidence cannot be silently converted into a pass or a failure.
3. **Traceable explanation.** Every flagged issue must point back to the exact value used by the rule.
4. **Controlled correction.** A reviewer needs guidance without allowing software to rewrite the source invisibly.
5. **Accountable history.** Runs, explanations, decisions, and corrected versions must remain reconstructable.

## 2.2 ClaimGuard's response

ClaimGuard converts every accepted package into one normalized scoring envelope and sends that envelope through a deterministic rule engine. The engine produces fifteen results, including explicit uncertainty and not-applicable outcomes. The evidence resolver records RFC 6901 paths and their exact values [3]. The workspace then places the issue, source evidence, rule text, explanation provenance, and corrective recommendation in one review context.

The optional AI layer is intentionally subordinate. It may explain why a result matters and propose what the reviewer should check. It cannot alter the result, edit the claim, record a review action, or submit anything. A verifier either accepts the output, allows one repair, or serves deterministic fallback text. This makes model failure visible without making it operationally decisive.



## 2.3 Scope and non-goals

The current system is a synthetic-data pilot. The rules are fictional, and agreement with the instructional oracle is not evidence of real reimbursement accuracy. A pass means that one fictional rule raised no objection. It does not mean that a payer will approve or pay the claim.

ClaimGuard does not make clinical judgments, assess medical necessity, identify fraud, or advise treatment. It does not transmit a claim to a payer. The Technical Manager workspace is deliberately content-blind. General PDF and image extraction, production-grade identity, database row-level isolation, and live monitoring integration are later hardening steps rather than hidden assumptions.

# Requirements and Principles

## 3.1 Phase 1 requirement coverage

Table 3.1 shows how the four scored requirements map to implemented components and reviewer-visible evidence.

**Table 3.1:** Phase 1 rubric coverage.

| Requirement                          | ClaimGuard implementation                                                                                                                                                      | Reviewer-visible evidence                                                                |
|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| Data ingestion and normalization     | JSON envelope, relational CSV, and FHIR R4 plus verified sidecar converge on the exact 17-key scoring contract. Invalid input becomes a structured rejected intake.            | Source type, intake state, normalized draft, error reason, and run link.                 |
| Deterministic and AI rule engine     | The versioned catalogue defines R001 to R015. The deterministic engine produces exactly fifteen results. AI is restricted to verified explanation and correction guidance.     | Reproducible rule results, status counts, explanation provenance, and labelled fallback. |
| Explainability and structured output | Each result carries Claim ID, Rule ID and version, severity, affected lines, evidence, source, explanation, corrective action, confidence semantics, method, and review state. | Finding, source value, rule, reason, and next step appear together.                      |
| Audit log engine                     | Runs and human decisions append events to a SHA-256 chain. Claim corrections create a later run version and preserve the first.                                                | Activity and audit views, version history, integrity status, and replay evidence.        |

## 3.2 Design principles

### 3.2.1 Deterministic authority

Only the rule engine establishes status, severity, evidence, and the requirement for human review. This decision is the most important architecture choice because it keeps the scored behavior reproducible and makes the optional model removable without changing any claim result.

### 3.2.2 Evidence before prose

A result is assembled from the rule outcome and evidence that re-resolves against the original envelope. Explanation follows this step. This order prevents fluent wording from becoming the source of truth

and makes unsupported citations detectable.

### **3.2.3 One internal contract**

Input adapters absorb source-format differences. The engine evaluates a single exact envelope rather than branching its logic for JSON, CSV, and FHIR. Missing facts stop at intake or remain explicit uncertainty; they are never guessed.

### **3.2.4 Versioning instead of overwriting**

A corrected claim creates a new run with a higher version and a link to the previous run. The earlier envelope, results, decisions, and audit evidence remain readable. This supports comparison, replay, and reviewer accountability.

### **3.2.5 Least authority for assistance**

The assistant reads the stored finding and a bounded evidence subset. It writes only append-only conversation turns. It has no code path to the claim result, reviewer decision, or payer boundary.

### **3.2.6 Tenant scope at the service boundary**

The browser chooses an appropriate workspace, but the API derives tenant and actor from the signed session. Reads and writes apply clinic scope again. The interface is therefore a usability layer, not the security boundary.

# ClaimGuard - Evidence-First Healthcare Claim Pre-Validation Architecture

Review before submission. Explain with evidence. Keep the human in control.

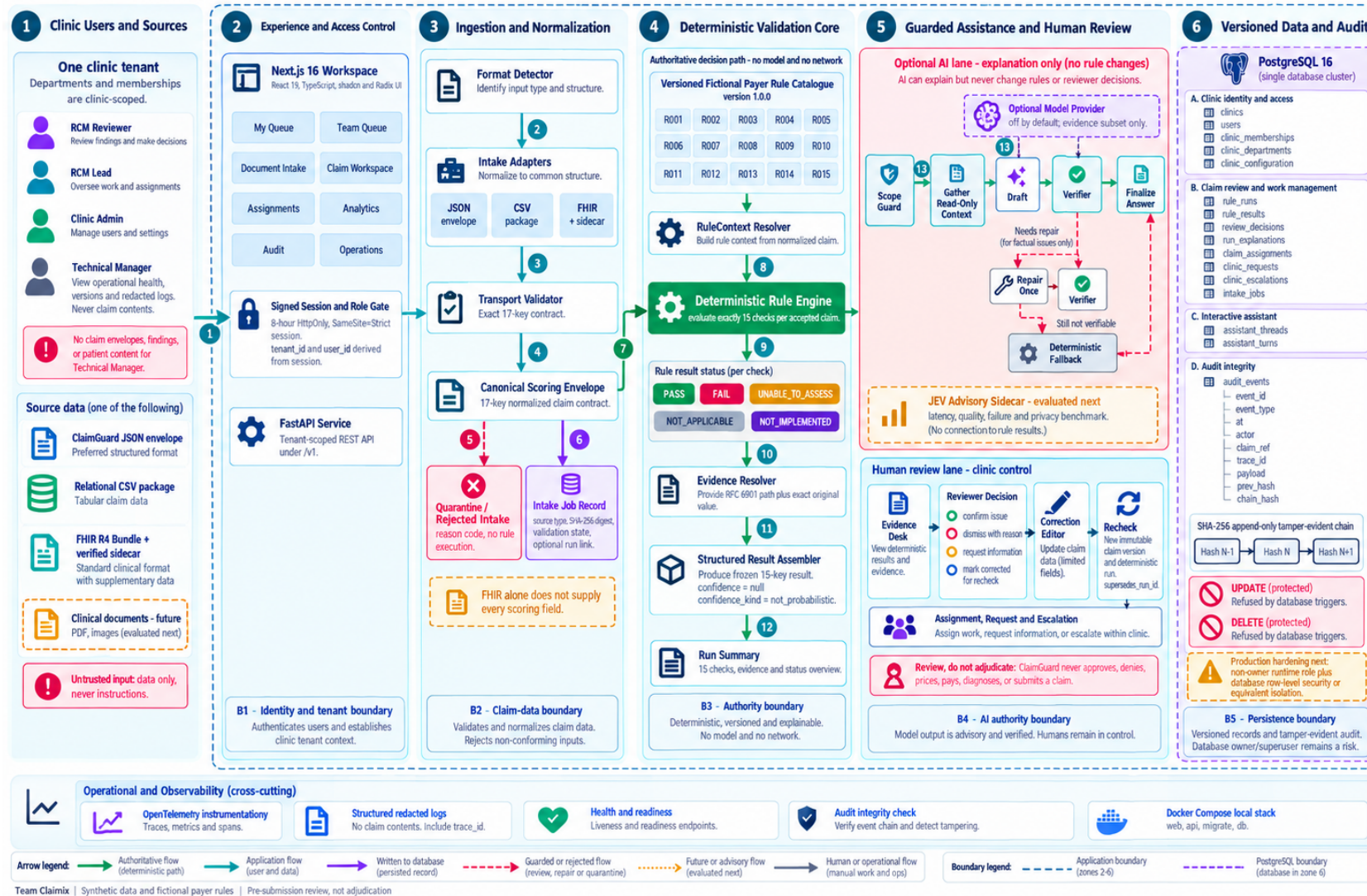


Figure 3.1: Global ClaimGuard source architecture. The plate is intentionally presented on a full landscape page to keep labels and trust boundaries readable.

# Global Source Architecture

---

Figure 3.1 organizes the platform into six runtime zones and one cross-cutting operational rail. The central green path is authoritative. Teal links represent application requests. Violet links represent persisted records. Rose links identify guarded, rejected, or security-sensitive behavior. Amber indicates future or advisory work.

## 4.1 Layer 1: clinic users and sources

One clinic is one tenant. An RCM Reviewer works assigned or self-submitted claims. An RCM Lead also manages assignments and escalations. A Clinic Admin manages the clinic directory, departments, access, routing, analytics, and audit views. A Technical Manager receives operational metadata but is blocked from claim envelopes and findings.

The supported structured sources are a complete ClaimGuard JSON envelope, a relational CSV package, and a FHIR R4 Bundle accompanied by a verified sidecar [2]. The sidecar is required because the current educational FHIR projection does not carry every leaf required by the scoring contract. General clinical documents are shown in amber because extraction from PDFs, images, invoices, or narrative reports remains a future proof of concept.

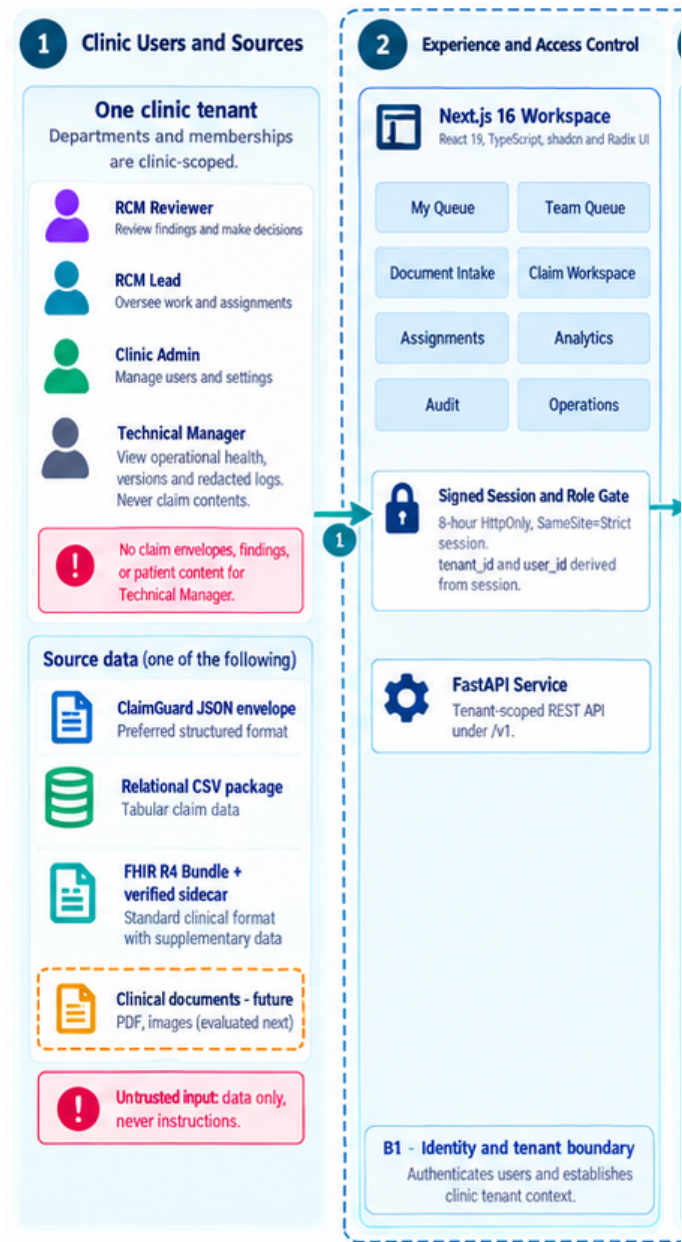


Figure 4.1: Enlarged view of the clinic roles, supported sources, workspace, and access-control boundary.

4.2 Layer 2: experience and access control

The Next.js workspace is the primary product surface. It exposes different navigation and actions according to role, but every protected request still passes through the FastAPI session gate. The session is HMAC-signed, HttpOnly, SameSite Strict, and limited to eight hours. Membership is resolved from current database state, so revoking a membership prevents later access.

The API derives `tenant_id` and `user_id` from the session rather than trusting request data. This prevents a caller from changing a tenant identifier in a body or query to cross clinic boundaries. The current implementation still relies on application-level tenant predicates. A non-owner runtime database role and row-level security or an equivalent isolation mechanism remain production gates [6].



### 4.3 Layer 3: ingestion and normalization

The format detector classifies the package before parsing it. It does not guess when a payload is incomplete. JSON, CSV, and FHIR plus sidecar then use dedicated adapters to produce the same candidate envelope. The transport validator checks the exact keys and types. A valid candidate becomes the canonical scoring envelope; an invalid one enters quarantine with a structured reason and does not reach any rule.

An intake job records the source type, SHA-256 source digest, state, validation information, and optional run link. The source digest supports provenance without claiming that the current intake pilot is a general-purpose clinical document archive.



**Figure 4.2:** Ingestion and normalization layer. Every accepted source converges on the 17-key scoring envelope.

4.4 Layer 4: deterministic validation core

The rule catalogue provides rule metadata and fictional policy context. The RuleContext resolver binds the normalized claim to versioned policies, service codes, provider lists, diagnosis catalogues, authorization conditions, required documents, limits, and submission windows. An unknown policy is not replaced with a default.

The engine evaluates R001 to R015 and always emits one result per rule. It distinguishes PASS, FAIL, UNABLE TO ASSESS, NOT APPLICABLE, and NOT IMPLEMENTED. A proven violation takes precedence over missing evidence, while missing evidence prevents an unjustified pass. The evidence resolver then records a pointer and the exact original value that supported the outcome. The result assembler creates the frozen structured record.

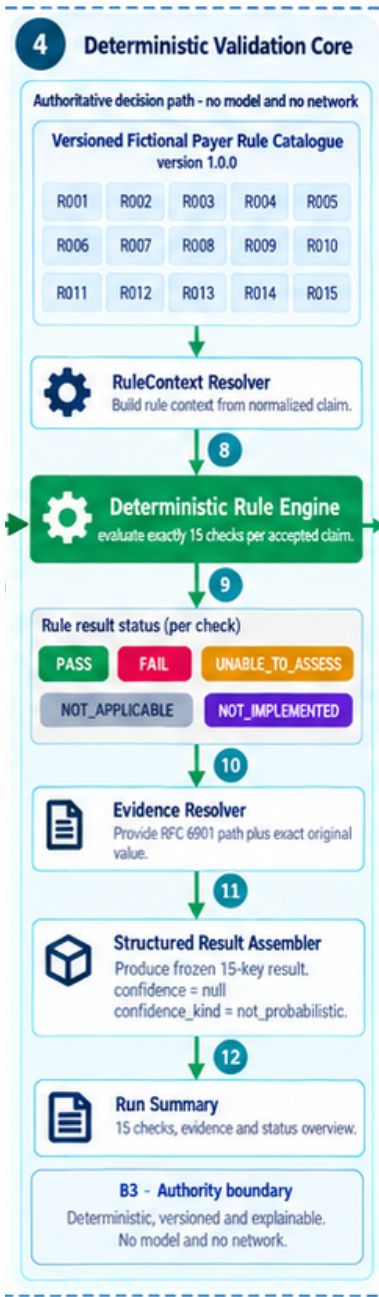


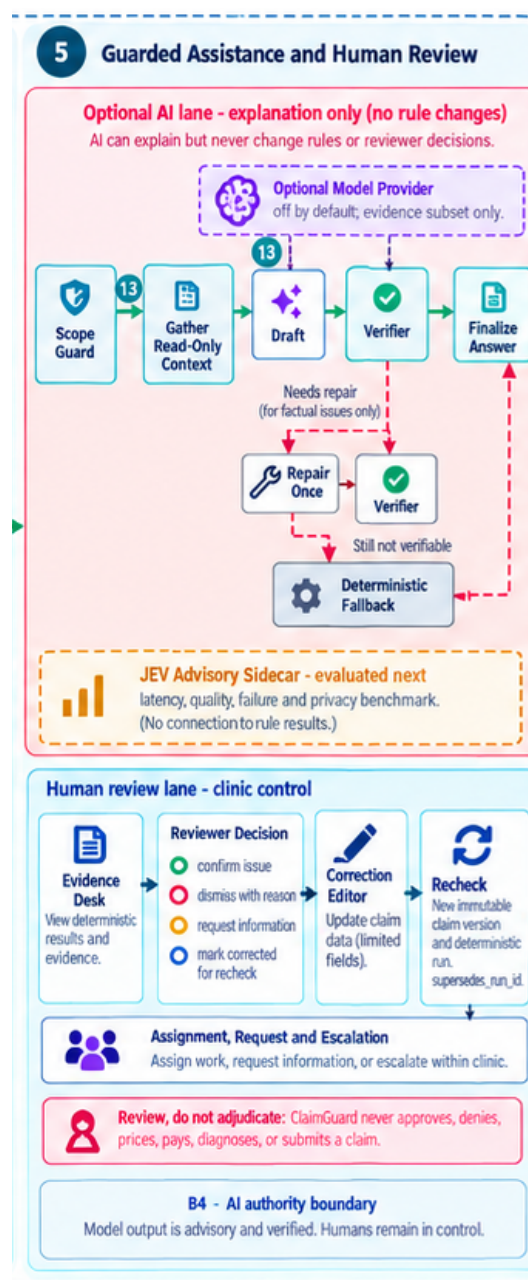
Figure 4.3: Authoritative validation core. No model and no external network are required to establish a result.



## 4.5 Layer 5: guarded assistance and human review

The upper lane is an optional explanation path. A deterministic scope guard runs before any model is reachable. It accepts only questions grounded in the active finding and refuses decision requests, clinical advice, injection attempts, and unrelated questions. The gather step reads the finding, cited evidence values, rule text, relevant policy limits, and the existing conversation. Drafting is optional. The verifier checks the exact answer schema, re-resolves citations, enforces allowed rule identifiers, and rejects prohibited assertions. One repair is allowed; otherwise deterministic fallback text is served.

The lower lane is the accountable workflow. The Evidence Desk aligns the finding with original values and guidance. The reviewer may confirm an issue, dismiss it with a reason, request information, or mark it corrected for recheck. A correction editor changes a limited set of fields under human control. Recheck creates a later immutable version and sends it through the same deterministic path. Recheck creates a later immutable version and sends it through the same deterministic path.



**Figure 4.4:** Guarded assistance and human review. The model path and reviewer authority remain separate.

### 4.6 Layer 6: versioned data, audit, and operations

PostgreSQL stores the clinic directory, review workflow, explanation provenance, assistant conversations, and audit ledger. The protected tables favor append-only or immutable behavior. The audit chain links each event hash to its predecessor so later modification becomes detectable. The design is tamper-evident, not deletion-proof against a database owner or superuser.

The cross-cutting operational rail includes OpenTelemetry instrumentation [5], structured redacted logs, health and readiness checks, audit-integrity verification, and the Docker Compose local stack. Grafana or another monitoring interface is a planned integration, not a component claimed as deployed.

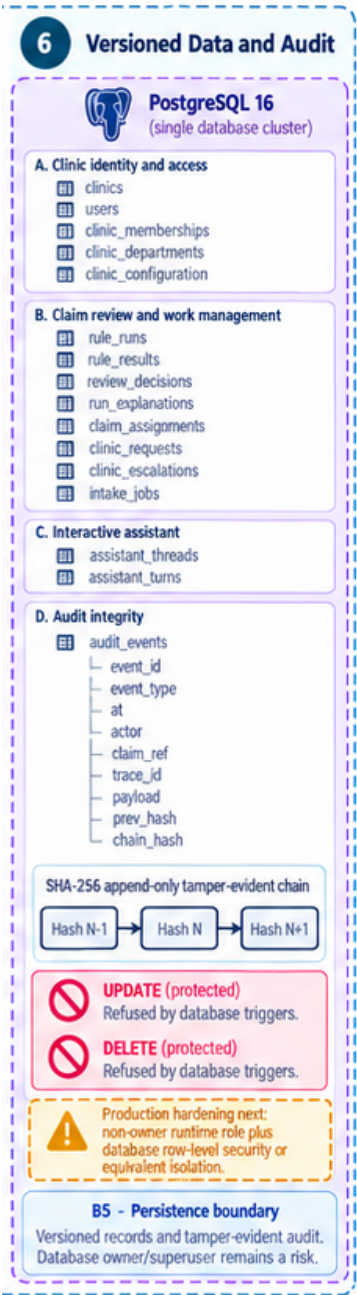


Figure 4.5: Versioned storage and audit boundary. Review records are tenant-owned and history-preserving.

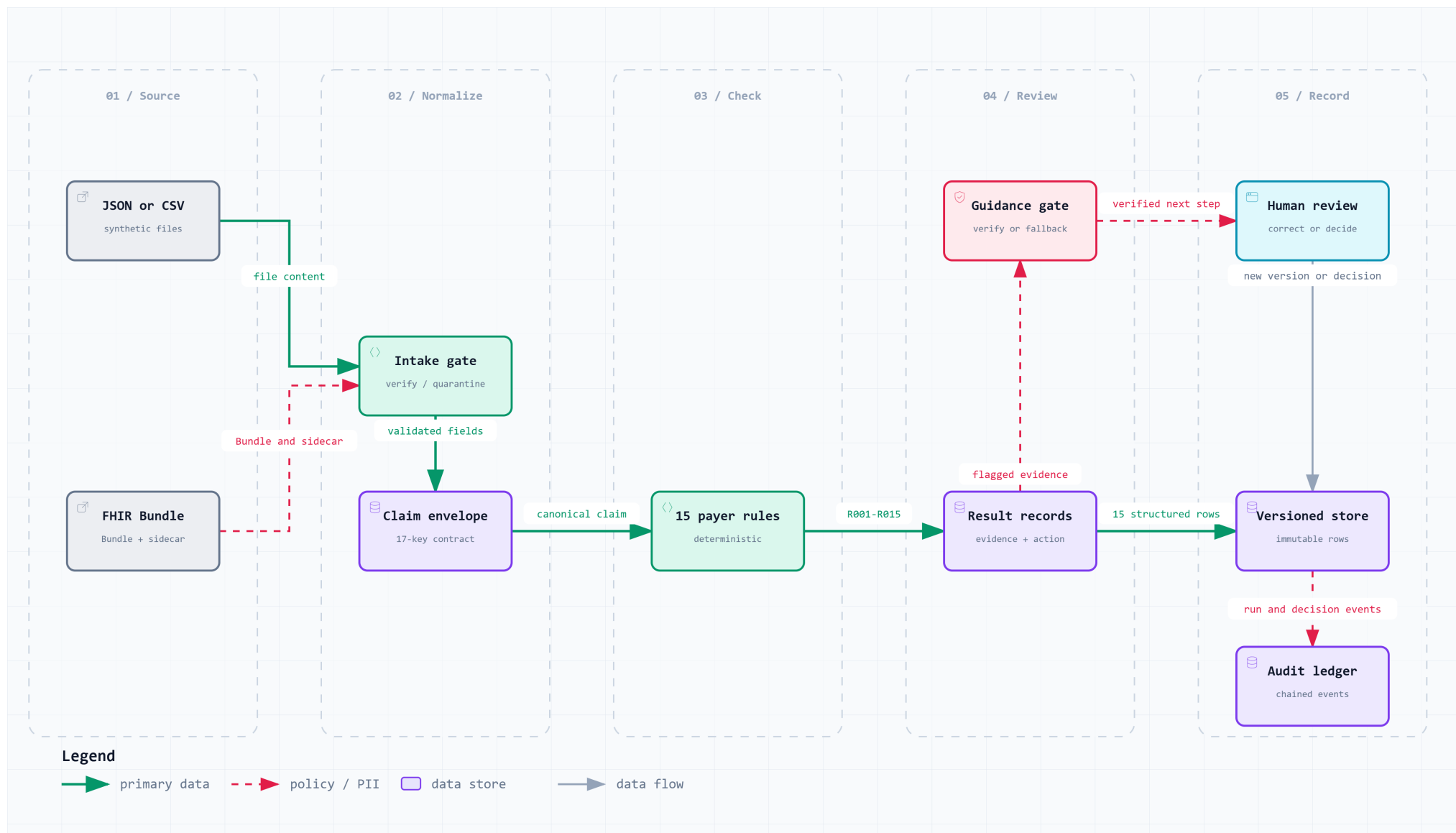
# Ingestion and Data Flow

---

## 5.1 The normalized claim contract

The scoring envelope contains exactly seventeen top-level keys. These keys cover schema identity, claim and invoice identifiers, patient and member identifiers, provider and payer identifiers, policy and diagnosis identifiers, submission date, currency, total amount, coverage, claim lines, authorizations, attachments, and notes. Nested structures retain the fields needed by the fictional rule catalogue.

Normalization does not mean filling gaps with plausible data. It means mapping known values into a stable structure and reporting unsupported or missing values. This distinction is essential for FHIR. The current mapping recovers a defined subset of envelope paths from the Bundle. The verified sidecar supplies the remaining scoring fields and is compared against every projected path before the package is accepted.



**Figure 5.1:** Claim package data flow from source through normalization, deterministic checks, review, versioning, and the audit ledger.

## 5.2 Flow interpretation

Figure 5.1 begins with a synthetic JSON or CSV package, or a FHIR Bundle accompanied by its sidecar. The intake gate validates rather than repairs. The canonical envelope becomes the only object evaluated by the rule engine. Exactly fifteen structured result rows are produced. Findings with attention-worthy outcomes can request a verified explanation, after which a human reviewer decides or corrects. Persisted versions and chained events complete the flow.

The red dashed paths in the figure do not represent a second decision system. They represent policy-sensitive or guarded information movement: the FHIR sidecar supplies missing contract fields, finding evidence constrains the explanation, and run and decision events enter the integrity chain.

## 5.3 Failure behavior

Malformed JSON, missing envelope keys, inconsistent FHIR sidecar values, invalid types, and unsupported shapes do not reach the engine. The API or batch process returns a structured ingestion error and records the failed intake where appropriate. This behavior prevents a parser defect from becoming a silent pass and keeps the reviewer informed about whether the system rejected a package or evaluated it.

# Deterministic Rules and Structured Explainability

## 6.1 Rule catalogue

The fifteen fictional payer rules cover completeness, chronology, eligibility context, provider and member consistency, duplicate detection, arithmetic, authorization, documentation, service support, policy limits, submission windows, and currency. Table 6.1 summarizes their intent.

**Table 6.1:** Fictional payer rules evaluated for every accepted claim.

| Rule | Check                     | Question answered                                                             |
|------|---------------------------|-------------------------------------------------------------------------------|
| R001 | Required information      | Are the mandatory claim and identity fields present?                          |
| R002 | Chronology                | Are service, coverage, authorization, and submission dates logically ordered? |
| R003 | Active coverage           | Was coverage active on the relevant service date?                             |
| R004 | Member consistency        | Do the member, beneficiary, and patient references agree?                     |
| R005 | Provider network          | Is the provider supported by the selected fictional policy?                   |
| R006 | Possible duplicate        | Do two lines represent a suspicious duplicate service entry?                  |
| R007 | Line arithmetic           | Does quantity multiplied by unit price agree with the line amount?            |
| R008 | Authorization reference   | Does a service that requires authorization cite one?                          |
| R009 | Authorization match       | Does the authorization match patient, service, date, status, and limits?      |
| R010 | Supporting document       | Is the required document present and in an acceptable state?                  |
| R011 | Service catalogue         | Is the service code supported by the fictional catalogue?                     |
| R012 | Claim total               | Does the total agree with the sum of line amounts?                            |
| R013 | Quantity and price limits | Are line quantity and price within fictional policy limits?                   |
| R014 | Submission window         | Was the claim submitted within the policy's allowed number of days?           |
| R015 | Currency                  | Does the claim currency match the policy currency?                            |

## 6.2 Five-state semantics

PASS means that the available evidence does not violate that fictional rule. FAIL means a violation is demonstrated. UNABLE TO ASSESS means required evidence is missing or insufficient. NOT APPLICABLE means the rule does not apply to the claim. NOT IMPLEMENTED is a distinct state that prevents unfinished logic from being presented as a pass.

Confidence is not invented for deterministic rules. The result uses a null confidence value and the explicit kind `not_probabilistic`. This is more truthful than assigning a number such as 1.0 to deterministic code and later treating it as calibrated probability.

## 6.3 Structured result record

Each result has fifteen fixed keys: claim identity, rule identity and version, status, severity, affected line identifiers, evidence, rule source, explanation, corrective action, confidence, confidence kind, human-review requirement, method, and review status. The scorer rejects missing or additional keys, unknown status values, incorrect rule version or source, and evidence whose stored value cannot be re-resolved from the submitted envelope.

This strictness improves the product interface. The UI does not need to infer which object belongs to which rule or whether a recommendation can be linked to a source. The same contract supports machine evaluation, reviewer display, persistence, replay, and later analysis.

## 6.4 Explanation and correction recommendation

The deterministic finding answers what happened. The assistance layer helps answer why the issue matters and what the reviewer should inspect next. Its output uses a small exact schema containing the explanation, correction recommendation, cited evidence paths, cited rule identifiers, and a human-review flag.

The verifier rejects fabricated evidence, foreign rule identifiers, unsupported adjudication or clinical claims, and instruction-shaped text. When the model is disabled or its output fails, deterministic wording remains available and the provenance says that fallback occurred. The reviewer can therefore distinguish accepted model wording from a safe twin.

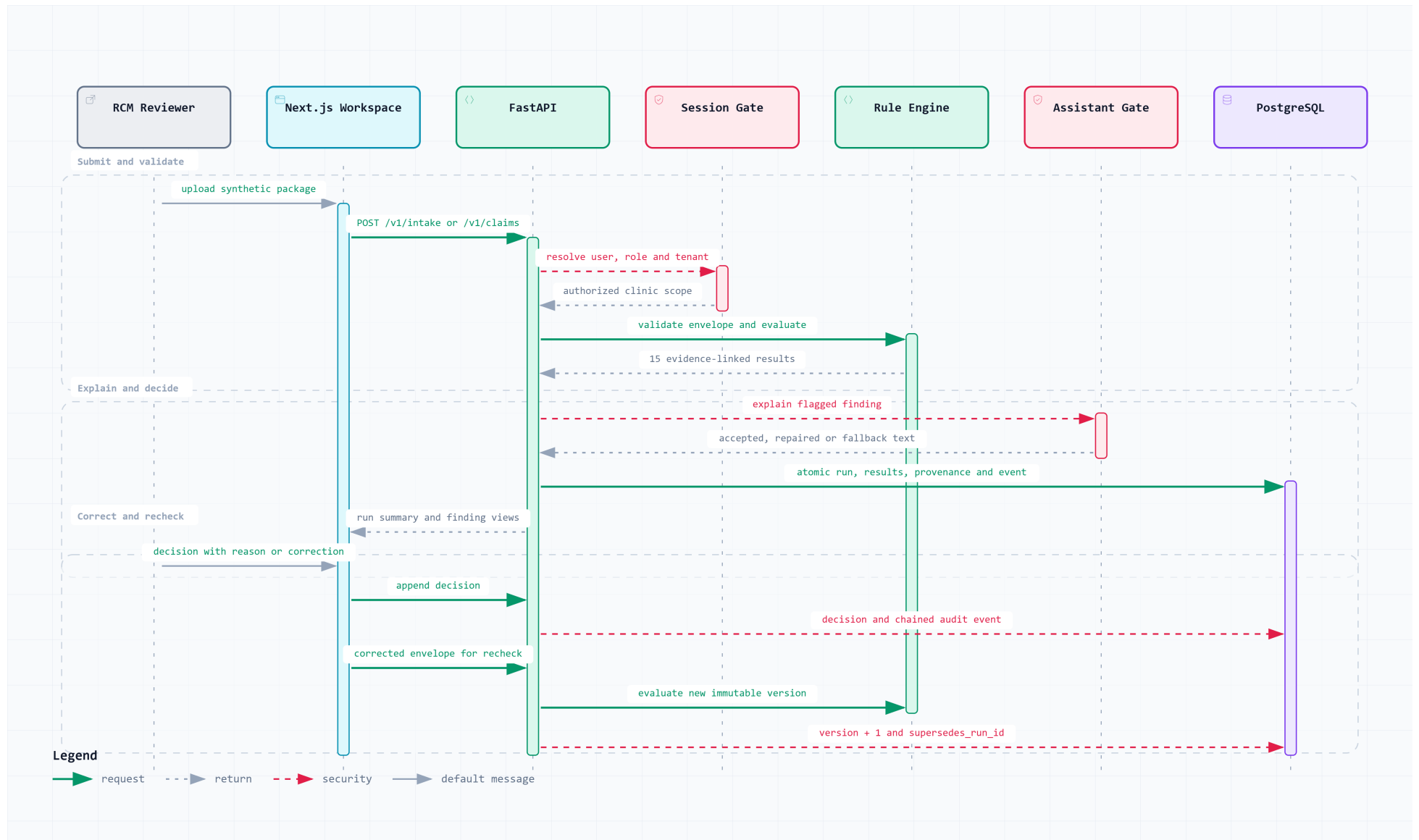
# Reviewer Lifecycle and AI Controls

---

## 7.1 One claim over time

The global architecture shows ownership, while a sequence diagram shows time. Figure 7.1 follows an accepted package through tenant authorization, rule evaluation, optional assistance, persistence, review, and correction. The database write stores the run, fifteen results, provenance, and event as one review record. A later decision appends rather than replacing the result.

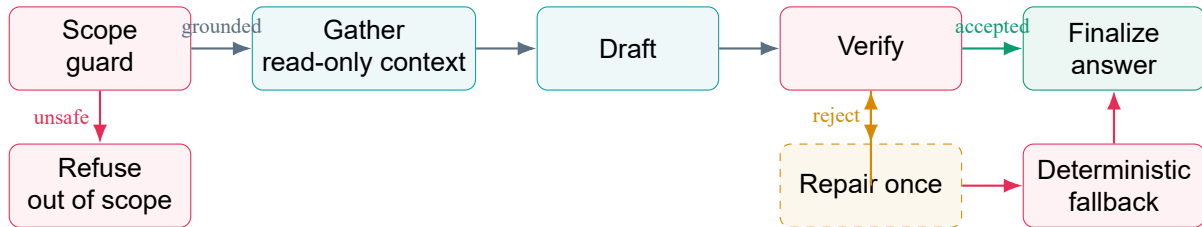




**Figure 7.1:** Sequence of claim submission, guarded explanation, reviewer decision, correction, and versioned recheck.

## 7.2 The interactive assistant as a bounded graph

The assistant is not a free-running agent. It follows a fixed graph in which only drafting and one repair may call a model. Every tool before the model is read-only, and every output after the model is verified.



**Figure 7.2:** Bounded assistant control graph. No node writes to the claim result or reviewer decision.

## 7.3 Why a small language model still needs this structure

A smaller model can reduce deployment cost and make self-hosting more practical, but size does not guarantee grounded behavior. Fine-tuning can improve style and domain vocabulary, yet it cannot replace runtime authority controls. ClaimGuard therefore evaluates model candidates on evidence fidelity, correction usefulness, refusal behavior, latency, and consistency. A candidate becomes deployable only when it passes those measurements without weakening deterministic fallback [8, 7].

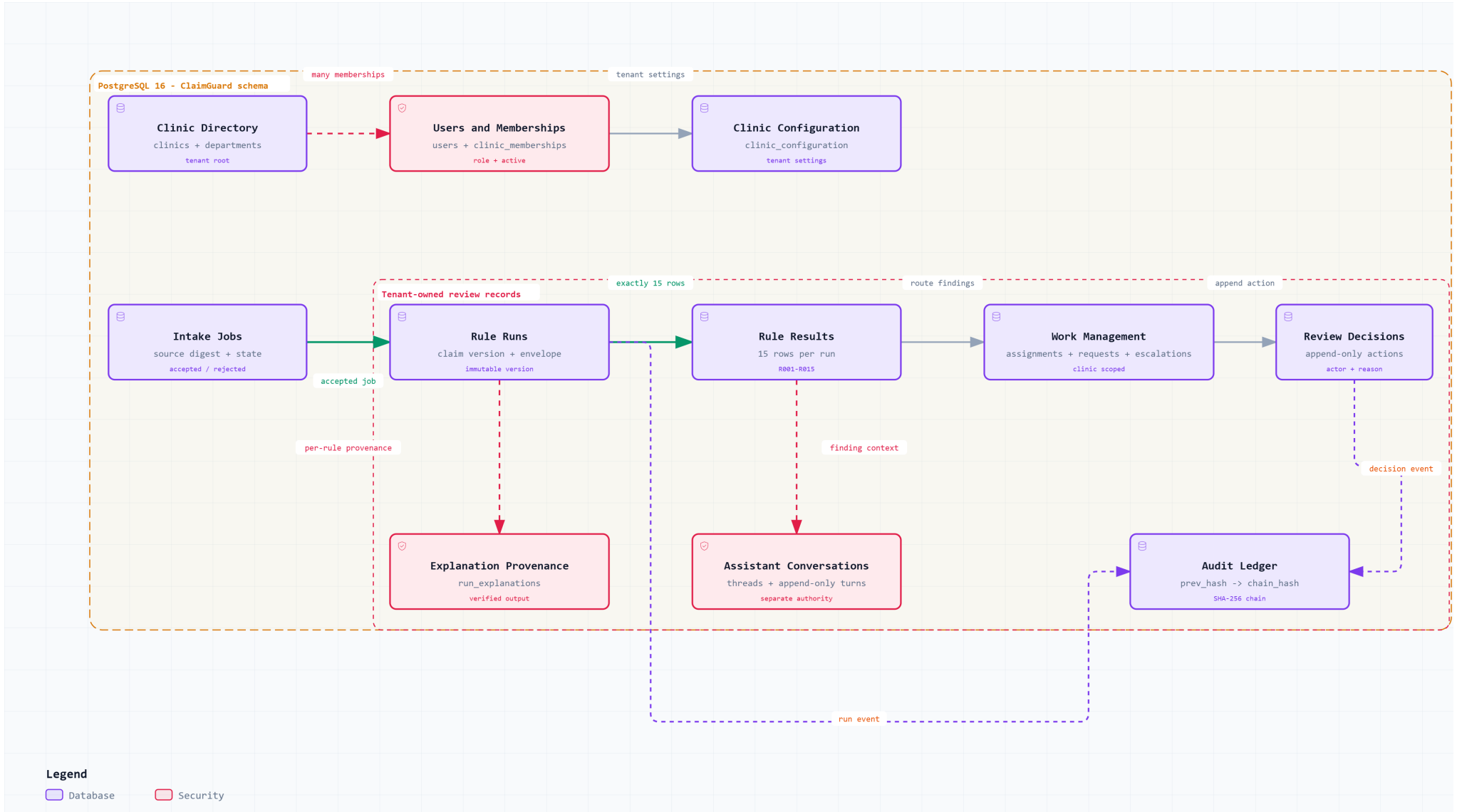
The benchmark and fine-tuning work remain separate from the Phase 1 authority path. The current default is model-off. That state is not a broken mode: the same interface serves deterministic explanation and identifies its provenance.

# Logical Data and Audit Architecture

---

## 8.1 Database organization

Figure 8.1 groups the PostgreSQL schema by responsibility [4]. Clinic identity tables establish tenants, users, memberships, departments, and configuration. Review tables preserve intake jobs, rule runs, results, assignments, requests, escalations, decisions, and explanation provenance. Assistant tables are separate so a conversation cannot become a rule result by accident. The audit ledger preserves the event sequence.



**Figure 8.1:** Logical PostgreSQL architecture and the principal relationships between tenant, run, result, workflow, assistant, and audit records.

## 8.2 Important cardinalities and invariants

**Table 8.1:** Principal data relationships.

| Relationship                            | Invariant                                                                                                                                  |
|-----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Clinic to membership</b>             | A clinic has many active or revoked memberships; the membership carries the role used by the API.                                          |
| <b>Run to result</b>                    | Every accepted run has exactly fifteen unique rule results, one for each R001 to R015.                                                     |
| <b>Run to decision</b>                  | A run can have many append-only decision events. Actor and reason are required.                                                            |
| <b>Run to explanation provenance</b>    | A per-rule record states whether model wording was accepted, repaired, declined, or replaced by fallback.                                  |
| <b>Run to corrected run</b>             | The newer run carries the next version and references the earlier run through <code>supersedes_run_id</code> .                             |
| <b>Run and rule to assistant thread</b> | A reviewer owns a conversation for one tenant, run, and rule. Other reviewers receive no existence disclosure.                             |
| <b>Thread to turn</b>                   | Turns are append-only, ordered by sequence, and carry model, prompt, verification, latency, and receipt information.                       |
| <b>Event to previous event</b>          | <code>prev_hash</code> links the event to the previous global ledger digest; <code>chain_hash</code> commits the current content and link. |

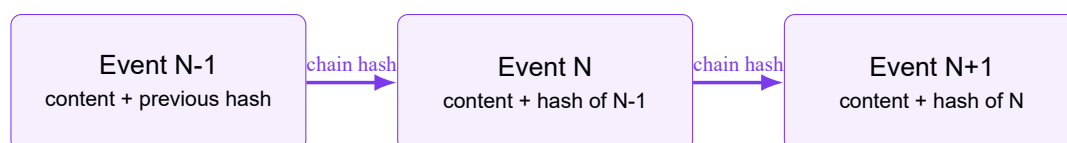
## 8.3 Tamper-evident audit chain

The database trigger computes each event's digest from a canonical serialization of its content and the previous chain hash. In conceptual form:

$$H_i = \text{SHA256}(H_{i-1} \parallel C_i)$$

where  $C_i$  is the normalized event content. Verification recomputes the sequence and reports a mismatch if content or linkage has changed. Run creation and reviewer decisions append events in the same transaction as their domain records. This prevents a successful decision write without its corresponding audit event under ordinary operation.

The ledger is accurately described as append-only and tamper-evident for the protected application path. A database owner can still remove triggers, truncate storage, or alter surrounding infrastructure. Production deployment therefore requires restricted ownership, backups, retention controls, off-system verification, and incident response.



**Figure 8.2:** Audit events are linked by content-dependent SHA-256 digests.

# Implementation and Verification

## 9.1 Technology stack

Table 9.1: Primary implementation technologies.

| Layer       | Technology                                             | Responsibility                                                                                                               |
|-------------|--------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Frontend    | Next.js 16, React 19, TypeScript, shadcn, Radix UI     | Role-specific workspaces, intake, evidence-led review, correction, administration, analytics, and operations.                |
| API         | FastAPI, Pydantic                                      | Tenant-scoped HTTP contracts, session enforcement, orchestration, structured errors, and OpenAPI documentation.              |
| Rules       | Python, Pydantic, JSON Pointer                         | Transport validation, versioned policy context, fifteen deterministic rules, evidence resolution, and frozen result records. |
| Assistance  | LangGraph-style bounded graph, optional model provider | Scope control, read-only context gathering, drafting, verification, one repair, fallback, and provenance.                    |
| Persistence | PostgreSQL 16, SQLAlchemy, psycpg, Alembic             | Tenant-owned records, versioned runs, append-only decisions, assistant turns, triggers, and audit chain.                     |
| Operations  | Docker Compose, Open-Telemetry, structured logging     | Reproducible local stack, health checks, traces, metrics hooks, and redacted logs.                                           |
| Quality     | pytest, Vitest, TypeScript, ESLint, Ruff, Pyright      | Engine, API, security boundary, UI, data contract, and conformance verification.                                             |

## 9.2 Verification evidence

The deterministic engine is tested against the public synthetic development, validation, and stress labels. Repository verification artifacts report full status agreement and issue detection on those public labels, including zero reported false alarms and zero missed issues. The same evidence must be interpreted narrowly: it demonstrates conformance to the fictional instructional oracle, not real-world clinical or reimbursement accuracy.

CSV reconstruction is checked against the normalized JSON form. FHIR behavior is tested as a declared projection rather than a claim of complete reconstruction. Evidence pointers are re-resolved by the scorer and independent conformance harness. Adversarial tests probe boundary dates, arithmetic tolerance, policy limits, hostile free text, unsupported provider and service cases, and model attempts to introduce

foreign evidence or prohibited conclusions.

The audit replay reconstructs a stored run from its envelope, re-evaluates all fifteen rules, compares the reproduced records, and verifies the global event chain. Assistance ablation verifies that model unavailability changes no deterministic status. These checks matter because a green unit-test suite alone does not prove scorer conformance, evidence fidelity, or replayability.

### **9.3 What the measurements do not prove**

The mentor-held claims are not available locally and therefore cannot be reported as evaluated. Public labels do not exercise every rule edge. Manual explanation-quality review is still needed for model-generated wording. The tenancy design has application-level enforcement but has not yet completed production row-level isolation. A successful local demonstration does not establish production privacy, availability, or legal compliance.

# Limitations and Next Work

---

## 10.1 Current limitations

- The dataset is synthetic and the rule catalogue is fictional.
- FHIR alone does not reconstruct every scoring field. A verified sidecar is required, and Encounter is not retained as a scored top-level field.
- General clinical-document extraction from PDF, image, or narrative sources is not authoritative.
- The optional model has not yet passed the complete grounded qualitative gate required for default deployment.
- Audit coverage is strongest for validation runs and reviewer decisions. Rejected intake, administrative changes, and every UI interaction are not all represented as immutable audit events.
- Application-level tenant filters require database-level hardening before real patient information is considered.
- Local passwords and sessions support the synthetic pilot. Production identity needs invitations or SSO, reset, throttling, CSRF review, and operational recovery.
- JEV remains an advisory sidecar under evaluation and has no authority over a rule result.

## 10.2 Prioritized next work

1. **Submission rehearsal and documentation.** Reproduce JSON, CSV, and FHIR plus sidecar intake; show one failed finding, a guided correction, a new-version recheck, and audit verification using synthetic data only.
2. **Held-out and human evaluation.** Run the mentor-held dataset when available. Score explanation and correction quality manually with evidence-fidelity and usefulness criteria.
3. **SLM decision.** Compare candidate models under the secured answer contract. Fine-tune only if measured errors show that prompting and retrieval cannot close the gap without weakening refusal or evidence fidelity.
4. **Document-to-draft proof.** Define a synthetic dental package with pre-authorization, invoice, visit notes, and attachments. Extract fields with provenance, conflicts, missing-value handling, and mandatory human confirmation before validation.
5. **Tenant and identity hardening.** Add a non-owner runtime role, row-level security or equivalent isolation, stronger account lifecycle, and concurrency tests.
6. **Audit expansion.** Record rejected intake and high-impact administrative changes, establish retention and backup rules, and define incident response for chain failure.
7. **Bounded JEV experiment.** Evaluate grounding, agreement, and attention signals at a single advisory point with latency, failure, privacy, and ablation evidence. Never place it in charge of pass, fail, correction, eligibility, or submission.
8. **Usability and operations.** Conduct reviewer testing, accessibility and keyboard audits, responsive checks, and monitoring integration with redacted logs and service-level indicators.



# General Conclusion

---

ClaimGuard demonstrates that explainable claim pre-validation does not require a generative model to become the authority. The system accepts supported synthetic packages, normalizes them into one strict contract, runs fifteen deterministic checks, cites the exact source evidence, and places verified guidance beside a human-controlled review workflow. Corrections create new versions, and run and decision events remain traceable through the audit chain.

The architecture's contribution is the separation of concerns. Input adapters handle format variation. The deterministic core establishes what is true under the fictional rulebook. The assistance layer improves understanding within explicit authority limits. The reviewer decides what action to take. PostgreSQL preserves versions and provenance. This separation keeps the platform useful when the model is unavailable and makes every later enhancement subject to the same evidence-first discipline.

The next phase is not to make the system more autonomous. It is to increase the quality of its evidence: evaluate on held-out claims, score explanation usefulness, prove document extraction with field-level provenance, harden tenant isolation, expand audit coverage, and test bounded advisory innovations such as JEV. The target remains unchanged: help the administrator prepare a stronger claim while keeping the human accountable and the system honest about what it knows.

# Cross-Boundary Data Contracts

## A.1 Seventeen-key claim envelope

|                              |                                                                                     |
|------------------------------|-------------------------------------------------------------------------------------|
| schema_version               | Contract version.                                                                   |
| claim_id,<br>invoice_number  | Claim and invoice identity.                                                         |
| patient_id,<br>member_id     | Patient and insurance-member identity.                                              |
| provider_id,<br>payer_id     | Provider and fictional payer identity.                                              |
| policy_id,<br>diagnosis_code | Policy and diagnosis context.                                                       |
| submission_date,<br>currency | Submission timing and monetary currency.                                            |
| total_amount                 | Declared total.                                                                     |
| coverage                     | Coverage identity, state, beneficiary, member, and effective dates.                 |
| lines                        | Service code, date, modifier, quantity, price, amount, and authorization reference. |
| authorizations               | Patient, service, state, dates, and quantity limits.                                |
| attachments                  | Type, patient, service, date, document state, and untrusted text value.             |
| notes                        | Untrusted free text treated only as data.                                           |

**Table A.1:** Top-level scoring envelope and nested responsibilities.

A.2 Fifteen-key result record

|                                |                                                                         |
|--------------------------------|-------------------------------------------------------------------------|
| claim_id                       | Claim that was evaluated.                                               |
| rule_id, rule_version          | Rule identity and version.                                              |
| status, severity               | Five-state outcome and configured severity.                             |
| affected_line_ids              | Claim lines involved in the result.                                     |
| evidence                       | RFC 6901 source paths and exact resolved values.                        |
| rule_source                    | Versioned fictional rulebook reference.                                 |
| explanation                    | Deterministic or verified wording.                                      |
| corrective_action              | Suggested reviewer-controlled next step.                                |
| confidence,<br>confidence_kind | Null deterministic confidence and explicit non-probabilistic semantics. |
| requires_human_review          | True for outcomes that must enter the review path.                      |
| method                         | Deterministic evaluation method.                                        |
| review_status                  | Initial review workflow state.                                          |

Table A.2: Frozen result record emitted once per rule.

# Workspace Responsibilities

Table B.1: Role boundaries and workspace responsibilities.

| Role              | Main pages                                                                                      | Responsibility and restriction                                                                              |
|-------------------|-------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| RCM Reviewer      | My Queue, Document Intake, Claim Workspace, Requests, Activity                                  | Reviews assigned or self-submitted claims, records reasoned actions, corrects fields, and requests recheck. |
| RCM Lead          | Team Queue, Assignments, Escalations, Review Quality, reviewer pages                            | Balances workload and handles escalations while retaining reviewer capabilities.                            |
| Clinic Admin      | Overview, All Claims, Assignments, Departments, Team and Access, Analytics, Audit               | Manages the clinic tenant, people, departments, routing, and clinic-wide operational views.                 |
| Technical Manager | Operations, Intake Jobs, Model and Rule Versions, Redacted Logs, Audit Integrity, Configuration | Maintains service health and configuration without access to claim contents or findings.                    |

# Bibliography

---

- [1] CSTAM 3.0 Velodoc Challenge, *First Submission Requirements: Claim Pre-validation*, competition brief, 2026.
- [2] Health Level Seven International, *FHIR Release 4, Version 4.0.1*, 2019. Available: <https://hl7.org/fhir/R4/>.
- [3] P. Bryan, K. Zyp, and M. Nottingham, *JavaScript Object Notation (JSON) Pointer*, RFC 6901, April 2013. Available: <https://www.rfc-editor.org/rfc/rfc6901>.
- [4] PostgreSQL Global Development Group, *PostgreSQL 16 Documentation*, 2023. Available: <https://www.postgresql.org/docs/16/>.
- [5] OpenTelemetry Project, *OpenTelemetry Specification*. Available: <https://opentelemetry.io/docs/specs/otel/>.
- [6] OWASP Foundation, *Authorization Cheat Sheet*. Available: [https://cheatsheetseries.owasp.org/cheatsheets/Authorization\\_Cheat\\_Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html).
- [7] OWASP Foundation, *LLM Prompt Injection Prevention Cheat Sheet*. Available: [https://cheatsheetseries.owasp.org/cheatsheets/LLM\\_Prompt\\_Injection\\_Prevention\\_Cheat\\_Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/LLM_Prompt_Injection_Prevention_Cheat_Sheet.html).
- [8] National Institute of Standards and Technology, *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*, NIST AI 100-1, January 2023. Available: <https://doi.org/10.6028/NIST.AI.100-1>.